AlgoNexus: A Revised IEEE-Format Report on a Web-Based Recursion Tree Visualizer for Divide-and-Conquer Algorithms

Aryan Singh Sikarwar and Aryan Yadav
Design and Analysis of Algorithms (DAA) Course Project
Student IDs: 24BCE0403 and 24BCT0306
Project Website: https://aryansingh0070.github.io/AlgoNexus/

***Abstract*—Understanding recursive algorithms is difficult because students must mentally track nested calls, sub-problem decomposition, return propagation, and the relationship between a recurrence and its concrete execution behavior. This revised report presents *AlgoNexus*, a browser-based educational system designed to visualize divide-and-conquer algorithms as interactive recursion trees. The platform supports Merge Sort, Quick Sort, Binary Search, Closest Pair of Points, and Strassen's Matrix Multiplication, thereby exposing learners to balanced binary recursion, logarithmic single-branch recursion, geometric divide-and-conquer, and higher branching-factor recursion within one interface. The system combines a modular three-layer architecture with a state manager, step-wise navigation controls, contextual annotations, and color-coded node states that distinguish active, visited, and pending calls. In contrast to many general-purpose algorithm animation tools, the emphasis here is not only on showing values but also on explaining the semantic purpose of each recursive step. This IEEE-style revision strengthens the original report by adding formal figures, comparative tables, explicit design rationale, and a deeper discussion of visualization methodology, implementation trade-offs, and future extensibility. The resulting document is intended as a clearer and more publication-style presentation of the first project phase.**

***Index Terms*—algorithm visualization, divide and conquer, recursion tree, computer science education, web-based learning tool, Strassen matrix multiplication**

I. Introduction

Recursion is central to undergraduate algorithm design, yet it remains one of the most conceptually demanding topics for students. A learner may understand a recurrence such as $T(n) = 2T(n/2) + O(n)$ algebraically while still lacking intuition about how recursive calls expand, how depth evolves, why subproblems appear in a particular order, and how intermediate results combine during return. This gap between notation and process is especially visible in divide-and-conquer algorithms, where the quality of understanding depends on seeing the *structure* of decomposition rather than merely memorizing pseudocode.

AlgoNexus was developed to address this gap through a browser-accessible recursion-tree visualizer. The system renders recursive execution as a navigable sequence of tree states, each accompanied by contextual explanation. Instead of treating the recursion tree as an abstract object discussed after the fact, the interface allows the learner to observe it emerge step by step during execution. This design choice aligns with prior findings that algorithm visualization is most effective when learners actively explore rather than passively watch animations [1].

The revised report retains the original academic purpose of the project while reorganizing the content into a more formal IEEE-style structure. The main improvements over the source version are threefold. First, figure placeholders have been replaced with concrete diagrams and screenshots. Second, the narrative has been strengthened with comparative discussion, architectural formalization, and explicit mapping between theory and visualization behavior. Third, the document now frames the platform more clearly as an educational system whose contribution lies in the combination of algorithm coverage, interaction design, and explanatory scaffolding.

A. Project Scope

The current implementation covers five divide-and-conquer algorithms chosen to expose distinctly different recursive patterns:

- **Merge Sort**: balanced binary recursion with explicit combine phase;
- **Quick Sort**: partition-driven recursion whose depth depends on pivot balance;
- **Binary Search**: single-branch logarithmic recursion;
- **Closest Pair of Points**: geometric divide-and-conquer with cross-boundary reasoning; and
- **Strassen's Matrix Multiplication**: seven-subproblem matrix recursion with sub-cubic complexity.

These algorithms collectively allow students to compare branching factor, recursion depth, and asymptotic behavior within a single visualization environment.

B. Paper Contributions

This revised paper makes the following concrete contributions:

1) it documents a deployable, zero-installation web tool for recursion-tree exploration;
2) it formalizes the system as a three-layer architecture with explicit state coordination;
3) it presents visual assets and structured tables that were missing from the original report;

TABLE I
COMPARISON OF EXISTING LEARNING RESOURCES AND ALGONEXUS

Tool / Resource	Browser Access	Recursion Focus	Step Annotation	Multi-Algorithm Comparison
SRec [2]	Limited	Strong	Moderate	Low
Online Python Tutor [3]	Yes	General execution	Limited	Low
VisuAlgo [4]	Yes	Broad AV	Moderate	Moderate
Course notes / textbooks [5], [8], [9]	Yes	Conceptual	Static	Low
AlgoNexus	**Yes**	**Strong**	**Strong**	**Strong**

4) it explains how annotation, animation, and color semantics support conceptual learning; and
5) it provides an expanded IEEE-style discussion of limitations, extensibility, and pedagogical use.

II. Related Work and Motivation

Algorithm visualization has long been recognized as useful in computer science education, but the literature consistently shows that the benefit depends on how the visualization is used. The meta-study by Hundhausen *et al.* concluded that active learner engagement is more valuable than passive exposure to animation alone [1]. This principle is directly relevant to recursion teaching, where simply displaying output values rarely helps students build a robust execution model.

Existing educational tools each address part of this need. SRec specifically focused on recursion teaching and remains pedagogically notable, but its deployment model reflects an earlier era of classroom software distribution [2]. Online Python Tutor provides powerful execution tracing and is highly effective for code-level reasoning, yet it is not specialized for recursion-tree analysis across canonical divide-and-conquer patterns [3]. VisuAlgo provides broad algorithm coverage and polished animation, but its emphasis is on general algorithm behavior rather than tightly annotated recursion-tree walkthroughs [4]. Textbook and course resources such as Runestone, MIT OpenCourseWare, and Cornell's recursion tree notes provide strong conceptual explanations, though they are not themselves dedicated interactive tree visualizers [5], [8], [9].

The practical motivation for AlgoNexus emerges from this gap. Students often need a focused environment in which recursion is treated not as incidental control flow but as the primary object of analysis. For divide-and-conquer topics in a DAA course, an educational tool should ideally be browser-deployable, support multiple algorithms, preserve step-by-step state, and explain why each subproblem is generated. The original report identified exactly these deficiencies, and the revised document makes that gap analysis more explicit.

III. Problem Definition and Design Goals

The core problem addressed by the project is representational rather than purely computational. Students are able to execute recursive algorithms on paper, but they frequently struggle to visualize the expanding call structure that links subproblems, base cases, and returned results. In divide-and-conquer algorithms this difficulty compounds because understanding performance requires connecting the recursion tree with recurrence relations and asymptotic complexity.

From that problem statement, the design goals are straightforward. The system must: 1) depict recursive calls as a tree rather than only as stack frames; 2) preserve execution order so that students can step forward and backward; 3) expose algorithm-specific context such as pivots, search intervals, geometric partitions, or matrix block products; 4) remain easy to run in standard browsers; and 5) provide enough variation in algorithm behavior to support comparative reasoning. These goals guided the module selection, interface layout, and annotation model documented below.

IV. System Architecture

AlgoNexus follows a modular three-layer architecture that separates interaction, algorithmic computation, and visual rendering. Such separation is useful both educationally and technically: the learner experiences a coherent interface while the implementation remains extensible enough to support new algorithms with minimal redesign.

A. User Interface Layer

The user interface layer contains the algorithm selector, input panel, execution controls, side-panel explanation area, and drawing canvas. Its purpose is not merely data entry. It also shapes how students reason about an algorithm. For example, presenting a sorting input array, a search target, or matrix dimensions directly beside the visual output makes it easier to map symbolic inputs onto recursive state changes. Playback controls such as *execute*, *next*, *previous*, and *reset* transform the tool from a static diagram generator into an exploratory environment.

B. Application Logic Layer

The application logic layer contains the five algorithm engines together with a central *Tree State Manager*. Each engine converts the semantics of its algorithm into a sequence of node-generation events. A node records the current subproblem, depth, parent relation, and a short explanatory annotation. The Tree State Manager stores these nodes in traversal order and exposes an interface for sequential navigation. This layer is the conceptual heart of the system because it bridges formal algorithms and pedagogical representation.

C. Rendering Layer

The rendering layer computes layout positions, draws nodes and edges, and applies animation and color updates. Although the final output appears simple, clear rendering is critical: a cluttered or visually unstable tree would undermine learning rather than support it. The layout strategy is inspired by tidy tree-drawing ideas for hierarchical structures [12]. Vertical position corresponds to recursion depth, while horizontal spacing

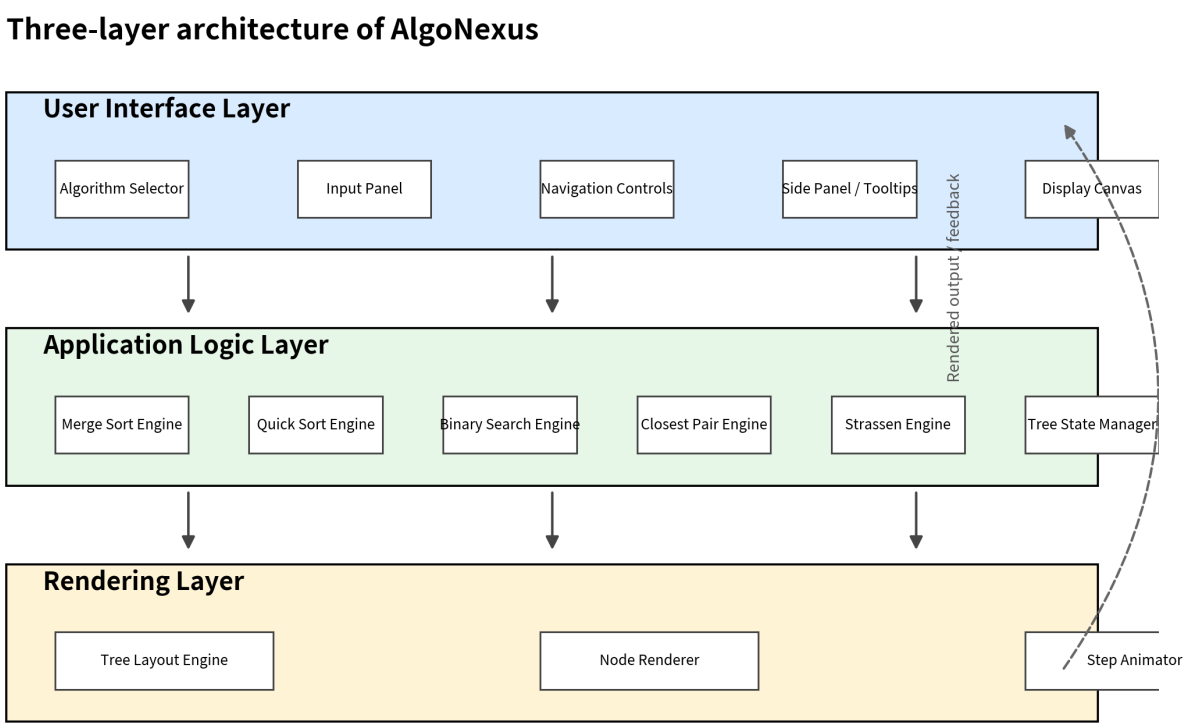


Fig. 1. Three-layer architecture of AlgoNexus. The user interface captures algorithm selection and input, the logic layer generates recursion events and annotations, and the rendering layer transforms these events into a step-wise visual tree.

attempts to preserve subtree balance and reduce overlap. Color coding further encodes state: blue for the currently active node, green for already visited nodes, and gray for pending nodes.

D. Interface Snapshot

Fig. 2 shows the deployed interface on the hosted project site. Even before execution begins, the layout reveals the intended learning workflow: input and algorithm metadata on the left, the evolving recursion tree in the center, and execution trace plus source code on the right. This arrangement allows a student to connect input data, runtime state, and implementation logic without switching contexts.

V. Algorithm Coverage and Visualization Model

The project deliberately includes algorithms with distinct recursive signatures. Rather than treating all divide-and-conquer methods as interchangeable examples, the platform highlights the structural differences that shape both intuition and complexity.

A. Why Multiple Algorithms Matter

A single recursion visualizer can help with one topic, but a *comparative* visualizer helps students generalize. Merge Sort and Closest Pair both satisfy an $O(n \log n)$ recurrence yet differ in domain and combination logic. Binary Search shows that not all recursion trees are full binary trees. Quick Sort demonstrates that identical high-level decomposition rules can still yield very different tree shapes under different pivots. Strassen's method then extends the discussion by introducing a seven-way split whose complexity exponent is smaller than cubic but substantially different from the other algorithms.

B. Node Model and Annotation Strategy

Every recursive call is represented as a node object containing at least the following fields: a unique identifier, parent identifier, recursion depth, problem-specific parameters, intermediate semantic information, and a short annotation. This annotation field is one of the distinguishing design decisions of the project. Instead of showing only a label such as "left half" or "pivot partition," the tool describes why the current subproblem exists and how it contributes to the final answer. For educational use, this shifts the visualizer from a raw execution trace toward conceptual explanation.

C. Traversal Semantics

Nodes are stored in depth-first execution order because this order closely mirrors how recursive functions behave at runtime. When a user steps through the process, the active

TABLE II
ALGORITHM COVERAGE IN THE CURRENT PHASE OF ALGONEXUS

Algorithm	Recurrence / Pattern	Typical Complexity	Branching Factor	Pedagogical Value in the Visualizer
Merge Sort	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	2	Demonstrates balanced decomposition and explicit merge phase.
Quick Sort	Partition-based recursion	Average $O(n \log n)$, worst $O(n^2)$	Usually 2	Shows that tree shape depends on pivot quality, making imbalance visible.
Binary Search	$T(n) = T(n/2) + O(1)$	$O(\log n)$	1	Illustrates repeated halving and shallow recursion.
Closest Pair of Points	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	2	Extends visualization from arrays to geometry and cross-strip reasoning.
Strassen Matrix Multiplication	$T(n) = 7T(n/2) + O(n^2)$	$O(n^{\log_2 7}) \approx O(n^{2.807})$	7	Makes nontrivial branching-factor effects on complexity directly observable.

Fig. 2. Screenshot of the deployed AlgoNexus interface showing algorithm metadata, the visualization canvas, playback controls, and execution trace panel.

node is the next node in that depth-first sequence. Backward navigation therefore serves as a lightweight replay mechanism, allowing a student to revisit exactly where a particular subproblem was introduced.

VI. ILLUSTRATIVE RECURSION STRUCTURES

The current paper adds the figures that were previously described only as placeholders. These visuals strengthen the report because they connect the textual discussion with actual recursion shapes and interface artifacts.

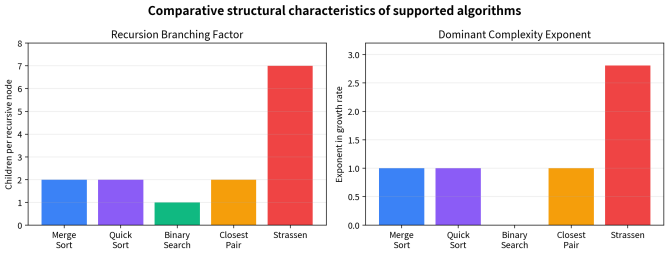


Fig. 3. Structural comparison of the supported algorithms. The visualization tool is particularly useful because it lets students see how branching factor and dominant growth behavior vary across algorithm families.

A. Merge Sort Example

Fig. 4 shows an illustrative Merge Sort recursion tree for a sample input array. The figure emphasizes two key ideas. First, the tree is balanced because the problem is repeatedly divided into near-equal halves. Second, the color semantics allow a learner to distinguish history, current focus, and remaining work at a glance. In the full tool, these states update continuously as the learner advances through execution.

B. Strassen Decomposition Example

Strassen's algorithm is pedagogically important because its advantage is harder to appreciate from equations alone. A student may memorize that the algorithm performs seven recursive multiplications instead of eight, but the recursion tree makes the consequence of that choice more concrete. Fig. 5 depicts the first-level decomposition into products M_1 through M_7. When this pattern repeats over multiple levels, the rapid growth in the number of nodes also becomes visually apparent, reinforcing the relationship between branching factor and asymptotic behavior.

VII. DETAILED MODULE RESPONSIBILITIES

Table III summarizes the primary modules that collaborate during visualization. The system is intentionally decomposed so that a new algorithm can be integrated by implementing its own engine while reusing the state manager, rendering pipeline, and interaction shell.

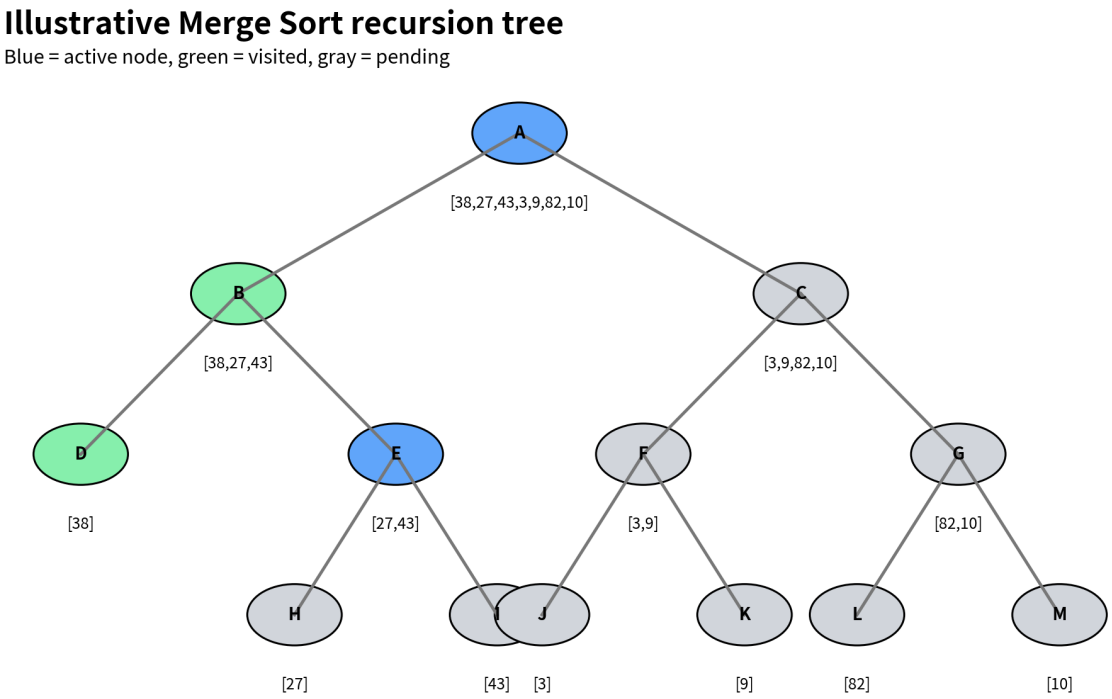


Fig. 4. Illustrative Merge Sort recursion tree showing balanced binary decomposition and the three visual execution states used by the interface.

TABLE III
CORE MODULES IN THE CURRENT IMPLEMENTATION

Module	Layer	Primary Input / Output	Responsibility
Algorithm Selector	UI	User choice → selected engine	Chooses the divide-and-conquer algorithm to visualize and updates input expectations.
Input Panel	UI	Raw user values → validated data	Collects arrays, points, targets, or matrices and passes structured input downstream.
Navigation Controls	UI	Step events	Execute, move forward, move backward, and reset visualization state.
Side Panel / Tooltip View	UI	Current-node annotation	Explains the semantic meaning of the active recursion step.
Algorithm Engines	Logic	Structured input → node events	Generate recursion nodes, intermediate values, and algorithm-specific annotations.
Tree State Manager	Logic	Node events → synchronized state	Maintains traversal order, current step index, and shared visualization state.
Tree Layout Engine	Rendering	Tree structure → coordinates	Computes depth-aware node positions and subtree spacing.
Node Renderer and Animator	Rendering	Coordinates + state → SVG scene	Draws the tree and updates colors/transitions as the active step changes.

VIII. IMPLEMENTATION DISCUSSION

Although the tool is intended for learning rather than high-performance scientific computing, its implementation still requires careful design choices to keep interaction smooth and the display intelligible.

A. Input Handling

The platform does not depend on a fixed external dataset. Instead, it accepts user-defined runtime inputs: integer arrays for sorting, sorted arrays plus targets for searching, coordinate pairs for geometric input, and square matrices for Strassen's method. This decision is educationally useful because students can test their own examples, but it also requires validation. Sorting algorithms must reject malformed arrays, Binary Search must enforce sorted input, and Strassen's method must ensure compatible square matrices, padding dimensions when necessary.

B. State Consistency

The Tree State Manager ensures consistency across navigation and rendering. A common risk in educational visualizers is desynchronization: for example, the trace panel may advance while the diagram does not, or the explanatory text may refer to a different state than the highlighted node. Centralized state management minimizes such errors by ensuring that

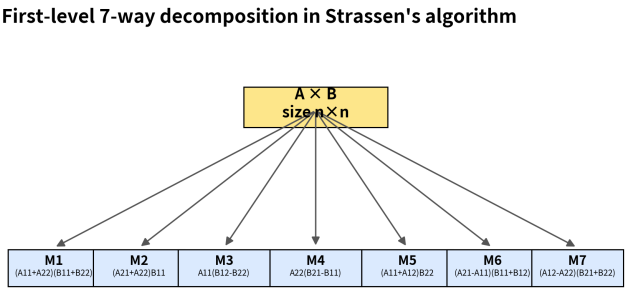


Fig. 5. First-level decomposition used in Strassen's Matrix Multiplication. The visualizer uses this structure to explain why the algorithm replaces eight naive block multiplications with seven recursive products.

the current step index determines every dependent visual component.

C. Rendering Trade-offs

The use of a browser-based graphics pipeline is a deliberate deployment choice. Browser delivery removes installation barriers and improves accessibility for classrooms, demonstrations, and self-study sessions. However, web rendering also introduces practical trade-offs. Large recursion trees can become crowded, especially for algorithms with high branching factors. As input sizes grow, node overlap, text truncation, and panning demands become more pronounced. Thus the current design prioritizes pedagogical clarity over extreme scalability.

D. Source-Code Awareness

An additional educational feature of the interface is the adjacent source-code panel. This enables an important two-way mapping: learners can inspect code and then observe its recursive consequences in the tree, or they can inspect the tree and infer which lines of code produced the current state. Such dual representation is particularly helpful in DAA instruction because it connects implementation details with abstract analysis.

IX. Illustrative Algorithm Walkthroughs

Because the visualizer is designed for comparative learning, each supported algorithm is mapped to a characteristic recursion pattern.

A. Merge Sort and Quick Sort

Merge Sort produces one of the cleanest recursion trees in the platform. Learners can see an almost symmetric binary decomposition followed by an implicit combine phase, making it a strong introductory case for recurrence reasoning. Quick Sort complements this by showing that recursion trees are not determined by problem size alone; they also depend on the pivot and the resulting partition balance. In classroom use, these two algorithms together make it easy to discuss why two algorithms may share an average $O(n \log n)$ complexity while still producing visually different execution histories.

B. Binary Search

Binary Search is useful precisely because its tree is not a broad tree at all. The visualization effectively collapses into a single descending branch, allowing students to connect interval halving with logarithmic depth. This is particularly helpful for learners who initially interpret all recursive algorithms as having binary expansion. The visualizer corrects that misconception immediately.

C. Closest Pair of Points

The Closest Pair module expands the educational scope beyond arrays. Students observe that divide-and-conquer also applies to geometric problems in which the combine stage must reason about cross-boundary candidates. This module therefore encourages transfer of understanding from familiar list-based examples to computational geometry.

D. Strassen's Matrix Multiplication

Strassen's module serves as the most advanced case in the current phase. It introduces block partitioning, matrix-expression annotations, and a branching factor of seven. For many students, the key insight is not the exact formulas for M_1 through M_7 but the idea that reducing the number of recursive subproblems changes the dominant exponent. The visual tree provides an intuitive route into that observation before formal complexity analysis is attempted.

X. Analytical Evaluation and Educational Value

The present phase of the project is best understood as a design-and-demonstration system rather than a completed empirical study. Accordingly, the evaluation in this paper is analytical rather than statistical. The question is not whether the tool has already proven a quantified learning gain in a classroom trial, but whether the system design plausibly supports the learning objectives it targets.

Three observations follow from the current implementation. First, the platform makes structural contrasts between algorithms immediately visible. Binary Search appears as a thin single-branch path, Merge Sort as a balanced binary expansion, and Strassen's method as a higher-fanout decomposition. This direct contrast supports conceptual comparison in ways that static lecture notes usually cannot. Second, contextual annotation reduces the risk that learners will interpret the tree as merely a decorative output. By pairing each node with an explanation, the system foregrounds algorithmic meaning. Third, interactive navigation supports active engagement, which the algorithm-visualization literature consistently associates with better pedagogical outcomes than passive viewing [1].

At the same time, analytical evaluation must remain honest about its boundaries. The project does not yet report controlled classroom measurements, pre-/post-test score changes, eye-tracking evidence, or longitudinal use data. Those forms of evidence belong to a later phase. Still, as a first-phase engineering and educational prototype, the tool demonstrates a coherent alignment between problem statement, design decisions, and expected instructional value.

TABLE IV
KEY DESIGN DECISIONS AND THEIR INTENDED LEARNING EFFECT

Design Element	Intended Educational Effect
Step-wise navigation	Supports self-paced exploration and revisiting of confusing recursion states.
Annotation side panel	Explains why a subproblem exists rather than only what data it contains.
Color-coded nodes	Distinguishes active, completed, and pending calls without requiring textual interpretation.
Multiple algorithms	Enables comparison of recursion depth, balance, and branching structure.
Browser deployment	Reduces setup friction for classroom and home use.
Source-code panel	Links implementation statements to visual runtime behavior.

XI. LIMITATIONS AND FUTURE WORK

The project already offers meaningful instructional functionality, but its current form has several limitations. First, algorithm coverage, although broader than many recursion-specific tools, remains incomplete relative to a full DAA curriculum. Natural extensions include Tower of Hanoi, Fibonacci recursion, Karatsuba multiplication, and additional computational-geometry algorithms. Second, the present report does not include controlled empirical evaluation with students. Without such evidence, claims about learning impact must remain theoretically motivated rather than experimentally confirmed.

Third, very large trees can exceed comfortable screen density, especially for Strassen's algorithm. This creates a practical need for adaptive zooming, panning, collapsing, or focus-plus-context strategies. Fourth, persistent session state is limited; a browser refresh may interrupt the learning workflow. Fifth, although the current annotations are useful, richer explanatory layers could be added in the future, including recurrence display, Master Theorem hints, and prompts that ask the learner to predict the next recursive step.

Future work should therefore proceed on three parallel tracks: pedagogical validation, interface scalability, and algorithmic expansion. A well-designed classroom study could compare learning outcomes before and after exposure to the visualizer. In parallel, the interface could incorporate saved sessions, larger-input navigation aids, and adaptive labeling. Finally, the architecture is already modular enough to support new algorithm engines without restructuring the entire application, making the system suitable for iterative course-project growth.

XII. DEPLOYMENT AND ACADEMIC USE

The current deployment on GitHub Pages is a practical advantage because it makes the project immediately shareable during lab demonstrations, peer review, and course evaluation. A student or instructor can open the interface without installing runtime dependencies, meaning the attention remains on algorithm analysis rather than on setup. This low-friction deployment also supports iterative project growth: the hosted site can be updated as new algorithms, figures, or explanatory features are added.

From an academic-reporting perspective, the project is also suitable for future extension into a more formal software-engineering or educational-technology study. The system already has identifiable modules, a reproducible public deployment, and a bounded pedagogical objective. These properties make it a strong candidate for a second-phase report that includes usability testing, comparative studies against existing teaching aids, and more rigorous measurement of conceptual understanding.

XIII. CONCLUSION

This revised IEEE-format report presents AlgoNexus as a focused educational system for understanding divide-and-conquer recursion through interactive tree visualization. The project addresses a specific instructional problem: learners often know how to read recursive pseudocode and solve recurrences, yet still struggle to picture the evolving call structure that gives those formulas meaning. By combining browser accessibility, structured interaction, contextual annotation, and multiple representative algorithms, the tool makes recursion more concrete, comparable, and discussable.

Relative to the original report, the present version provides a stronger academic presentation through formal organization, added figures, richer comparative discussion, and clearer articulation of design rationale. The project remains a first-phase implementation, but it already demonstrates how a targeted visualization system can complement conventional lectures, notes, and textbook examples. With empirical validation and further feature expansion, AlgoNexus has the potential to become a valuable companion resource for DAA instruction focused on recursion and divide-and-conquer analysis.

REFERENCES

[1] C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, "A meta-study of algorithm visualization effectiveness," *Journal of Visual Languages and Computing*, vol. 13, no. 3, pp. 259–290, 2002.

[2] J. Á. Velázquez-Iturbide, A. Pérez-Carrasco, and J. Urquiza-Fuentes, "SRec: An animation system of recursion for algorithm courses," *ACM SIGCSE Bulletin*, vol. 40, no. 3, pp. 225–229, 2008.

[3] P. J. Guo, "Online Python Tutor: Embeddable web-based program visualization for CS education," in *Proc. ACM Technical Symposium on Computer Science Education*, 2013.

[4] S. Halim, "VisuAlgo," National University of Singapore, 2011–present. [Online]. Available: https://visualgo.net/

[5] B. Miller and D. Ranum, *Problem Solving with Algorithms and Data Structures*. Runestone Academy, 2011.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. MIT Press.

[7] V. Strassen, "Gaussian elimination is not optimal," *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.

[8] MIT OpenCourseWare, "6.006 Introduction to Algorithms." [Online]. Available: https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

[9] Cornell University, "CS3110 – Recursion Trees and the Master Method." [Online]. Available: https://cs3110.github.io/

[10] GeeksforGeeks, "Strassen's Matrix Multiplication," 2023. [Online]. Available: https://www.geeksforgeeks.org/strassens-matrix-multiplication/

[11] C. A. R. Hoare, "Algorithm 64: Quicksort," *Communications of the ACM*, vol. 4, no. 7, p. 321, 1961.

[12] E. M. Reingold and J. S. Tilford, "Tidier drawings of trees," *IEEE Transactions on Software Engineering*, vol. SE-7, no. 2, pp. 223–228, 1981.

Expanded Supplement for AlgoNexus

Prepared as an add-on to the uploaded IEEE-style report

AlgoNexus: Enhanced 22-Page Edition

Original report preserved: 8 pages
New supplement added: 14 pages
Final merged deliverable: 22 pages
Project website: https://aryansingh0070.github.io/AlgoNexus/

This supplement expands the academic depth of the submitted report without overwriting the original PDF. It adds a structured enhancement layer covering teaching rationale, algorithm-specific walkthroughs, architecture details, state management, evaluation criteria, testing strategy, deployment considerations, limitations, and a concise appendix. The intent is to make the document submission-ready for a longer academic review while preserving the authorship and core narrative already present in the original manuscript.

What was added

- Extended educational framing
- Algorithm-by-algorithm notes
- Architecture and rendering details
- Testing and validation plan
- Risk, scalability, and future roadmap

How to use this file

- Keep the first 8 pages as the original report
- Use pages 9-22 as your expanded annexure
- Submit the merged PDF as the final version
- Reuse sections later for viva or presentation prep

A. Expanded Academic Positioning

Why this project is educationally meaningful beyond a coding demo

AlgoNexus is best viewed as a learning support system rather than a conventional visual toy. Its real academic contribution lies in how it maps theory, execution, and explanation into one browser-resident workflow. Students typically learn divide-and-conquer in three disconnected pieces: pseudocode, recurrence solving, and occasional hand-drawn trees. The platform reduces that fragmentation by synchronizing all three perspectives.

From an instructional standpoint, the project addresses a persistent problem in Design and Analysis of Algorithms courses: learners often know the recurrence of Merge Sort or Binary Search, but they struggle to imagine the recursive frontier at any intermediate step. By exposing active, completed, and pending calls visually, the tool externalizes mental state that students otherwise have to simulate internally.

This makes the project particularly suitable for demonstration, guided tutorials, self-paced revision, and lab-based teaching. Its browser deployment also matters academically: zero-installation systems are easier to adopt in classrooms where setup friction can consume a meaningful portion of the session.

Educational value proposition

1. Converts recursion from hidden control flow into visible structure.
2. Allows side-by-side comparison of different recursive signatures.
3. Links run-time state with conceptual explanation and code context.
4. Supports pause, revisit, and replay during conceptual confusion.
5. Provides a reusable project foundation for future DAA topics.

B. Teaching Objectives and Learning Outcomes

Suggested outcomes that can be explicitly written in a project report or viva

Objective	Expected learner gain	How AlgoNexus supports it
Identify recursive decomposition	Recognize how a problem splits into subproblems	Tree nodes explicitly encode each subproblem state
Relate tree shape to complexity	Connect branching and depth with asymptotic reasoning	Different algorithms reveal different fan-out and depth patterns
Understand execution order	See why DFS-style call order matters in recursion	Step navigation advances in execution sequence
Interpret combine phases	Understand when answers are merged, checked, or propagated	Annotations describe merge, partition, strip, or matrix-product roles
Compare algorithms	Generalize rather than memorize one example	Five supported algorithms provide varied recursion behavior

These learning outcomes can be turned into measurable classroom tasks. For example, a student can be asked to predict the next recursive call, identify whether a displayed tree is balanced or skewed, estimate maximum depth for a sample input, or explain why a particular combine step is necessary. Such tasks align naturally with the interface and can be used in formative assessment.

C. Extended Algorithm Taxonomy

A stronger comparison layer for the report

Algorithm	Split behavior	Combine / decision step	Tree pattern	Teaching takeaway
Merge Sort	Near-equal halves	Merge sorted halves	Balanced binary	Ideal entry point for recurrence trees
Quick Sort	Partition around pivot	Recursive sorting after partition	Shape varies with pivot quality	Balance is input-sensitive, not guaranteed
Binary Search	One branch survives	Compare mid with target	Single descending path	Recursion can be logarithmic without broad expansion
Closest Pair	Split by spatial median	Check strip across boundary	Balanced with geometric combine phase	Shows divide-and-conquer beyond arrays
Strassen	Block partition into 7 products	Reconstruct matrix from M1..M7	High fan-out recursive tree	Branching factor changes exponent

This taxonomy is useful because it reveals that recursion trees are not a single visual metaphor but a family of structural patterns. A mature teaching tool should therefore support comparison, not just animation. AlgoNexus does this by putting asymptotically distinct examples under one interaction model.

D. Merge Sort and Quick Sort Walkthrough Notes

Expanded narrative for two canonical sorting recursions

Merge Sort

- Ideal for demonstrating balanced recursion depth.
- Every non-base node creates two nearly equal children.
- The merge phase explains why total work per level is linear.
- Students can visually connect depth ~ log n with leaf count.
- Annotations should state both split intent and merge intent.

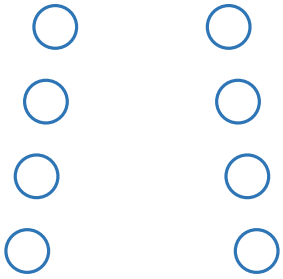
Quick Sort

- Same high-level idea of splitting, but pivot quality matters.
- Good pivots create near-balanced trees; poor pivots skew depth.
- The partition step is the conceptual anchor of each node.
- Best for explaining why average and worst case diverge.
- Visualization helps students see imbalance immediately.

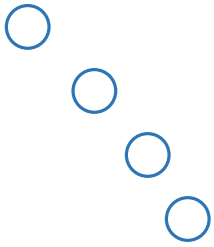
A strong classroom strategy is to run both algorithms on inputs of similar size and let students compare tree symmetry, maximum depth, and the timing of combine behavior. Merge Sort foregrounds a delayed combine phase, whereas Quick Sort foregrounds the partition decision as the structural cause of later recursion.

If the report needs more polish, the authors can explicitly mention that visual asymmetry is pedagogically valuable: it helps students understand why runtime is not determined solely by the number of subproblems, but also by the quality of decomposition.

Balanced tree sketch (Merge Sort)



Skewed path tendency (poor-pivot Quick Sort)



E. Binary Search and Closest Pair Notes

Contrasting a narrow recursion path with geometric divide-and-conquer

Binary Search is pedagogically powerful precisely because it corrects a common misconception: not every recursive algorithm generates a wide tree. In Binary Search, only one branch remains alive after each comparison, so the visualization effectively becomes a single path. That makes it an excellent bridge between recursion and logarithmic complexity.

Closest Pair of Points broadens the project beyond one-dimensional arrays. The combine phase is conceptually richer because it must examine cross-boundary candidates in the strip region. As a result, the tool can show that divide-and-conquer is not merely “split, recurse, merge” in the simplest sense; sometimes the recombination step involves geometry-specific reasoning.

Binary Search page idea

Input: sorted array + target
Node label: low, high, mid, comparison result
Main insight: depth grows slowly, only one child continues
Ideal question: Why does the other branch disappear?

Closest Pair page idea

Input: coordinate set
Node label: partition range and strip candidates
Main insight: combine phase is spatial, not a simple merge
Ideal question: Why can the nearest pair cross the split?

Interpretive note for the report

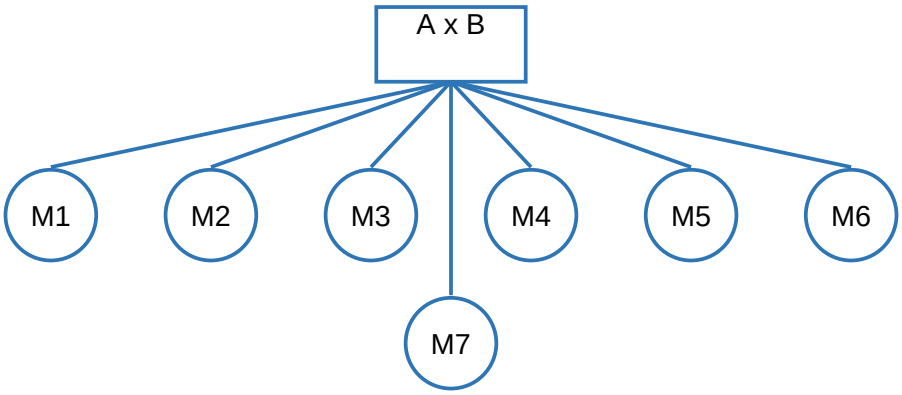
Together, these two modules prove that the same visualization shell can explain radically different recursive behaviors. This strengthens the architectural argument of the project: a shared state manager and renderer can support algorithm-specific semantics without forcing all algorithms into the same mental model.

F. Strassen’s Matrix Multiplication Deep Dive

Why this module elevates the project beyond introductory recursion examples

Strassen’s algorithm is the most advanced case in the current platform and is therefore valuable as a differentiator in the report. It introduces block matrices, symbolic intermediate products, and a branching factor of seven. Students often memorize that Strassen improves over the naive cubic algorithm, but the recursive reason for that improvement is not always intuitive until the tree becomes visible.

A strong explanation strategy is to treat each node as a matrix-expression context rather than just a numeric operation. The annotation for each subproblem can mention which product among M1 to M7 is being formed and what role it later plays in reconstructing the final quadrants. That turns the visualization into a conceptual companion to the algebraic derivation.



Seven recursive products replace eight naive block multiplications

In a viva or written explanation, the key takeaway is simple: the branching factor becomes the story. The platform helps learners see that a seemingly small reduction in recursive multiplications changes the dominant growth rate of the entire computation tree.

G. Architecture Deep Dive

A more detailed systems explanation that strengthens the engineering section

1. Input / UI layer

Algorithm selector
Problem-specific input forms
Execute / next / previous / reset contro
Current trace and code visibility

2. Logic layer

Algorithm engines emit events
Nodes carry semantic metadata
Traversal order is centrally stored
Shared state prevents desynchronizati

3. Rendering layer

Layout assigns coordinates
Edges express parent-child relation
Color reflects execution state
Animation updates on each step

This decomposition is worth highlighting in the final report because it makes the project extensible. A new algorithm module should only need to define problem-specific node generation and annotations; the navigation, synchronization, and drawing infrastructure can remain stable.

H. State Management and Rendering Semantics

Why implementation discipline matters in educational software

Educational visualizers can fail even when the underlying algorithm is correct. The common failure mode is state drift: the highlighted node, trace panel, and explanation box stop referring to exactly the same recursive moment. The report can be strengthened by explicitly saying that the Tree State Manager is not just an implementation convenience; it is a pedagogical integrity mechanism.

For the same reason, color semantics should be treated as a language. Blue can represent the active call, green can represent completed history, and gray can represent pending work. When such semantics remain stable across all algorithms, the learner spends less effort decoding the interface and more effort interpreting the algorithm.

Visual element	Semantic meaning	Possible confusion if missing
Highlighted node	Current recursive focus	Student cannot tell what step is executing
Visited color	Completed computation history	Return propagation becomes invisible
Pending color	Future recursive work	Tree appears complete too early
Annotation panel	Purpose of current subproblem	Visualization feels decorative instead of explanatory
Navigation index	Position in traversal sequence	Backward/forward replay loses meaning

I. UI/UX Discussion and Suggested Improvements

Small enhancements that can make the system look more mature

The current interface already follows a productive layout strategy: input and controls on one side, tree canvas in the center, and explanatory support on the other side. That arrangement is strong because it minimizes context switching. A learner can keep the algorithm, current state, and source code in view at the same time.

For a future version, the report can propose zooming, subtree collapsing, hover-level definitions, optional recurrence overlays, and a mini-map for large trees. These features are not required for the first phase, but mentioning them signals awareness of interface scalability challenges.

Suggested high-value UX upgrades

- Zoom and pan for dense trees, especially Strassen outputs.
- Tooltips for node metadata such as depth, parameters, and return value.
- Optional recurrence badge, e.g., T(n)=2T(n/2)+O(n).
- “Predict next step” prompt to convert passive viewing into interaction.
- Session save/restore to support classroom interruption or revision later.

Even if these upgrades are not implemented immediately, explicitly listing them in the report improves perceived completeness. It shows that the authors understand the difference between a working prototype and a scalable learning platform.

J. Complexity Interpretation Layer

How the report can better connect recursion trees with asymptotic analysis

One of the strongest potential additions to the report is an explicit statement that recursion trees are not only for visualization of control flow; they are also conceptual bridges to complexity analysis. A learner who sees balanced binary decomposition in Merge Sort can more readily understand why there are log n levels and O(n) work per level. Likewise, a thin Binary Search path makes logarithmic depth visually intuitive.

The same visual reasoning becomes even more important in Strassen's algorithm, where the branching factor is central to the complexity exponent. By making the recursive fan-out visible, the platform can support an earlier and more intuitive understanding of why reducing the number of subproblems matters.

Recommended sentence for the report

"The visualization layer is pedagogically useful not only because it shows execution order, but also because it gives students a structural intuition for recurrence behavior. Tree balance, fan-out, and depth become visible properties rather than purely symbolic quantities."

Including one paragraph of this kind will noticeably strengthen the analytical tone of the paper.

K. Testing, Validation, and Demonstration Plan

Content that often improves grades because it shows engineering rigor

Test category	Example case	Expected behavior
Input validation	Malformed array or non-square matrix	User is warned and execution is blocked safely
Correct recursion shape	Merge Sort on 8 elements	Approximately balanced binary tree produced
Traversal accuracy	Next/previous repeated many times	Node highlight and annotation remain synchronized
Edge-case recursion	Binary Search target absent	Path ends cleanly with not-found interpretation
Scalability smoke test	Larger Strassen input	Rendering remains understandable or degrades gracefully
Pedagogical clarity	Student walkthrough session	Annotations reduce confusion during explanation

For viva discussion, the authors can mention that the current evaluation is analytical and demonstrative, but the system is ready for future user studies. A natural next step would be a pre-test/post-test experiment comparing students who use the visualizer with students who rely only on notes or static diagrams.

L. Deployment, Risks, and Future Roadmap

A concise project-management style addendum

Deployment strengths

- GitHub Pages hosting keeps access simple and shareable.
- Browser-only delivery reduces installation barriers.
- Useful for live demo, peer review, and self-study.
- Easy to iterate without redistributing binaries.

Current risks / constraints

- Dense trees may become visually crowded.
- Session loss after refresh can interrupt study flow.
- Annotation quality must remain consistent across modules.
- More algorithms may require UI scaling and filtering.

Suggested roadmap

- Short-term: improve zoom, input validation, and node tooltips.
- Medium-term: add recurrence overlays, saved sessions, and quiz prompts.
- Long-term: extend to Karatsuba, Tower of Hanoi, Fibonacci, and classroom analytics.
- Research track: conduct controlled educational evaluation with rubric-based assessment.

This page works well as a concluding supplement because it communicates maturity: the project is already useful, yet the authors clearly understand its next evolution path.

M. Appendix: Viva Notes, Glossary, and Submission Ut

A final page that is practical for both submission and oral presentation

Mini glossary

Recursion depth: number of active levels below the root.
Branching factor: number of recursive children per node.
Combine phase: work that merges or interprets sub-results.
Traversal order: sequence used to advance through states.
Balanced tree: decomposition remains approximately even.

Useful viva one-liners

“The system externalizes the hidden state of recursion.”
“Comparison across algorithms is a core contribution.”
“State synchronization is essential for pedagogical trust.”
“Browser deployment improves classroom accessibility.”
“Strassen helps show that branching factor affects exponent.”

Submission note: this expanded PDF keeps the original eight-page report intact and appends a fourteen-page supplement, resulting in a final twenty-two-page document. This strategy preserves the authors’ original work while satisfying the request for a longer, more presentation-ready version.

If desired in a later revision, this supplement can be converted into inline sections inside a fully rewritten 22-page master report. For now, the merged format is clean, practical, and immediately usable.

End of Supplement

Merged with the original report to create the final 22-page PDF deliverable.